Urbanization from the Bottom Up

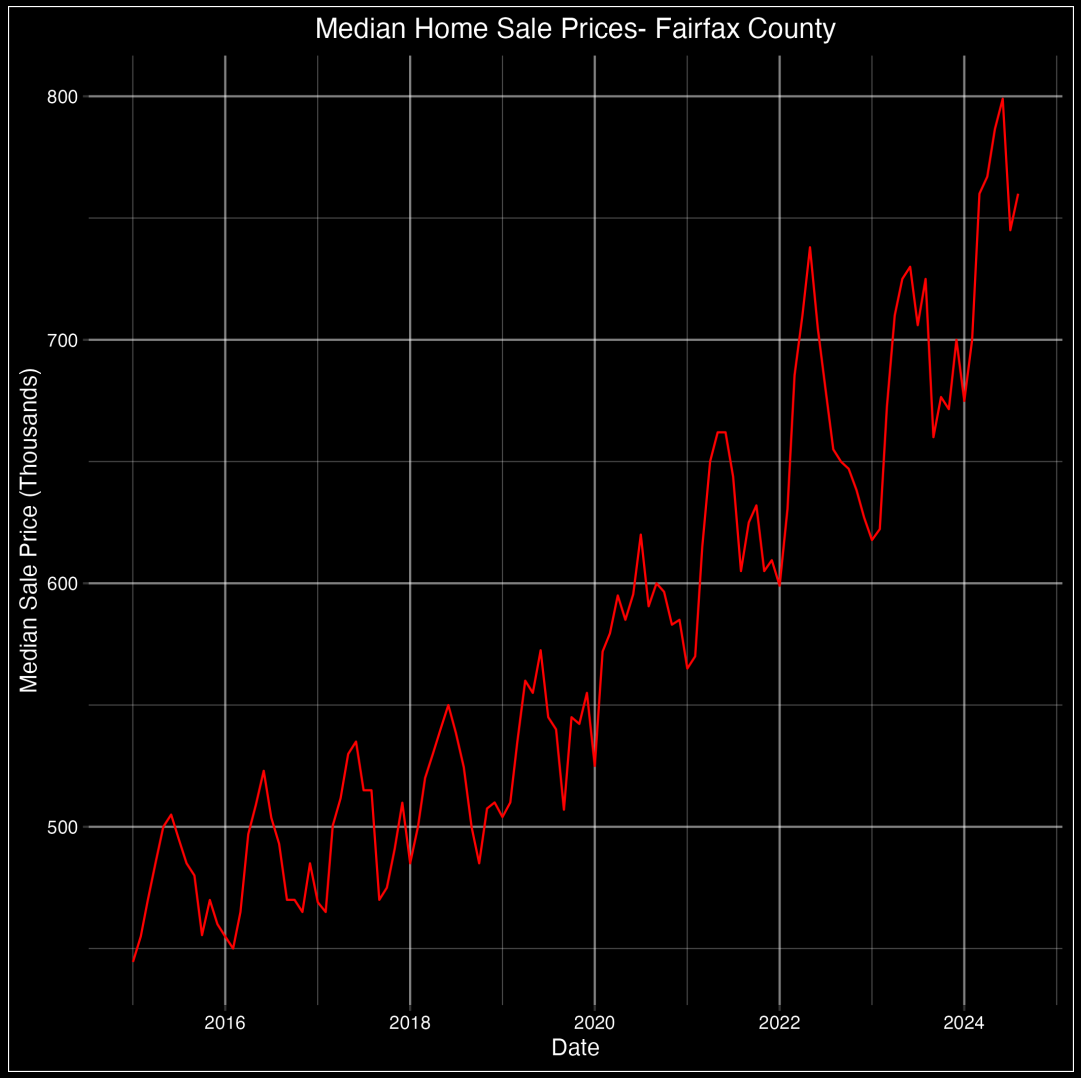
John S. Schuler
Department of Computational and Data Sciences
George Mason University

March 20, 2026

Outline

1. Empirical motivation: what happened to housing?
2. The NIMBY ABM: model design and mechanics
3. Building with generative AI
4. Parameter sweeps: what we learn overnight
5. Connections and future work

Fairfax County Home Prices



Why Agent-Based Modeling?

Standard models assume:

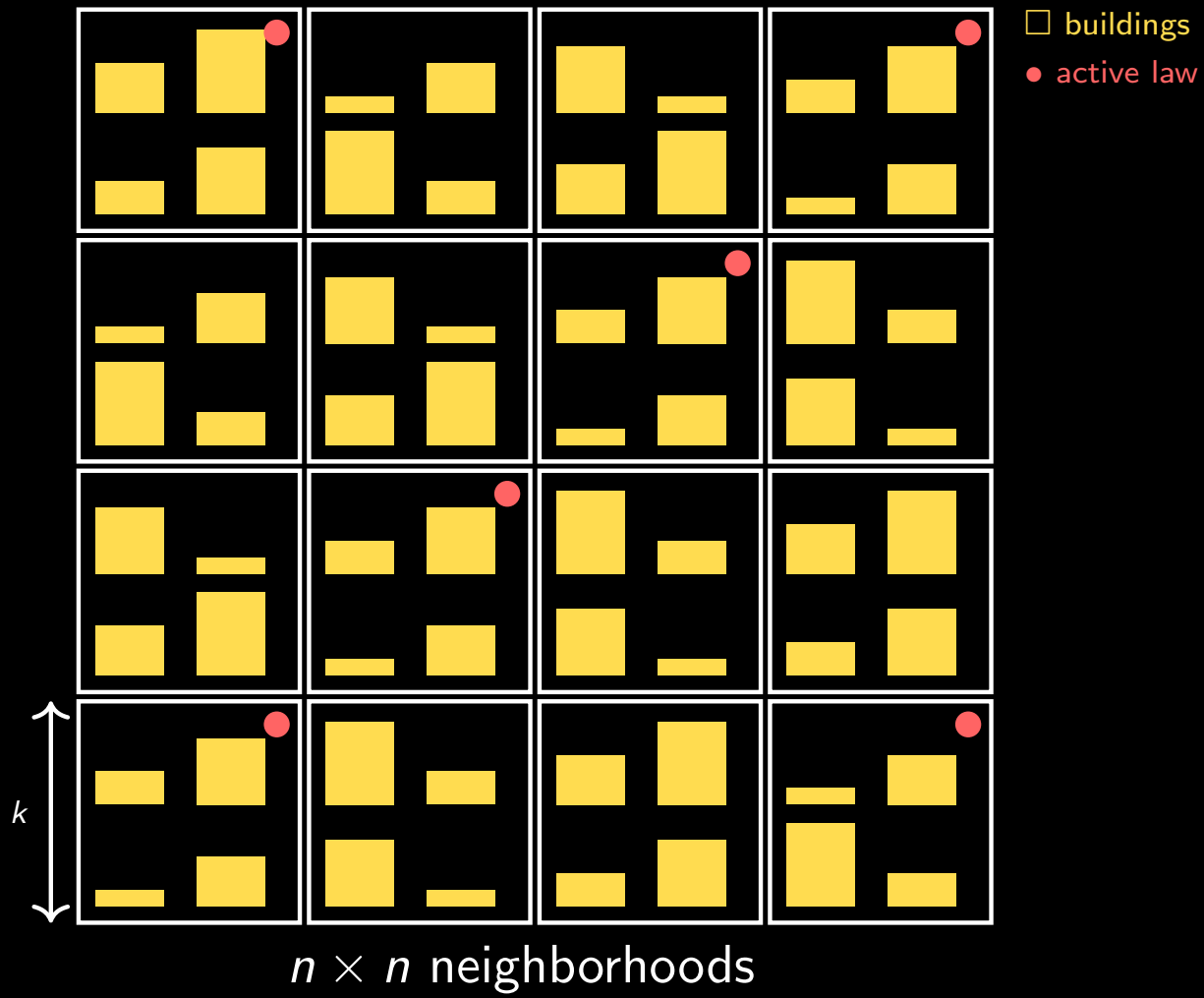
- Homogeneous agents
- Perfect information
- Instantaneous market clearing
- Zoning as an exogenous constraint

Housing is fundamentally:

- Heterogeneous (location, type, vintage)
- Locally governed (neighborhood politics)
- Path-dependent (who is already there)
- Endogenous supply (residents vote on zoning)

ABM lets residents, buildings, and laws co-evolve.

City Structure



Default: 10×10 neighborhoods, each 8×8 buildings $=$ **6,400 buildings**. Building *height* $=$ number of stacked dwellings. Neighbors vote on zoning laws.

Agent Preferences

Each agent i has a job location and residential preferences drawn from log-normal / truncated-normal distributions:

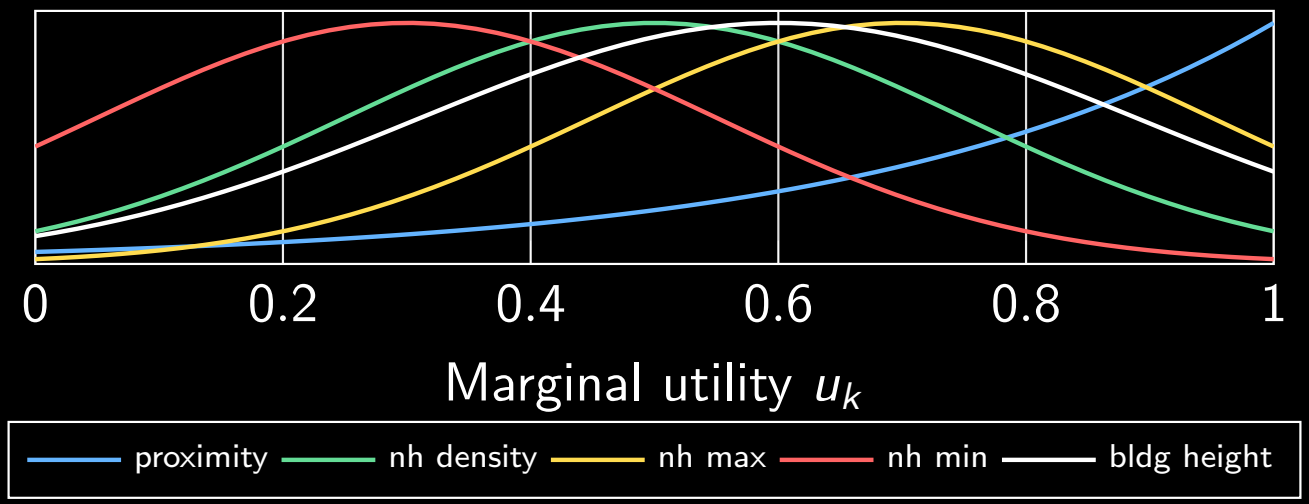
Parameter	Meaning	Distribution
b_i	budget	log-normal
$\bar{d}_i$	preferred neighborhood density	truncated normal
$\bar{h}_i^{\max}$, $\bar{h}_i^{\min}$	preferred max/min heights	truncated normal
$\bar{h}_i$	preferred building height	truncated normal
σ_i^{nh}, σ_i^{b}	sensitivity to deviations	log-normal
λ_i	proximity decay scale	log-normal
θ_i	Frank copula dependence	log-normal

Job buildings assigned at creation, weighted by $w(b) = \text{height}(b) + 1$.

The Utility Function

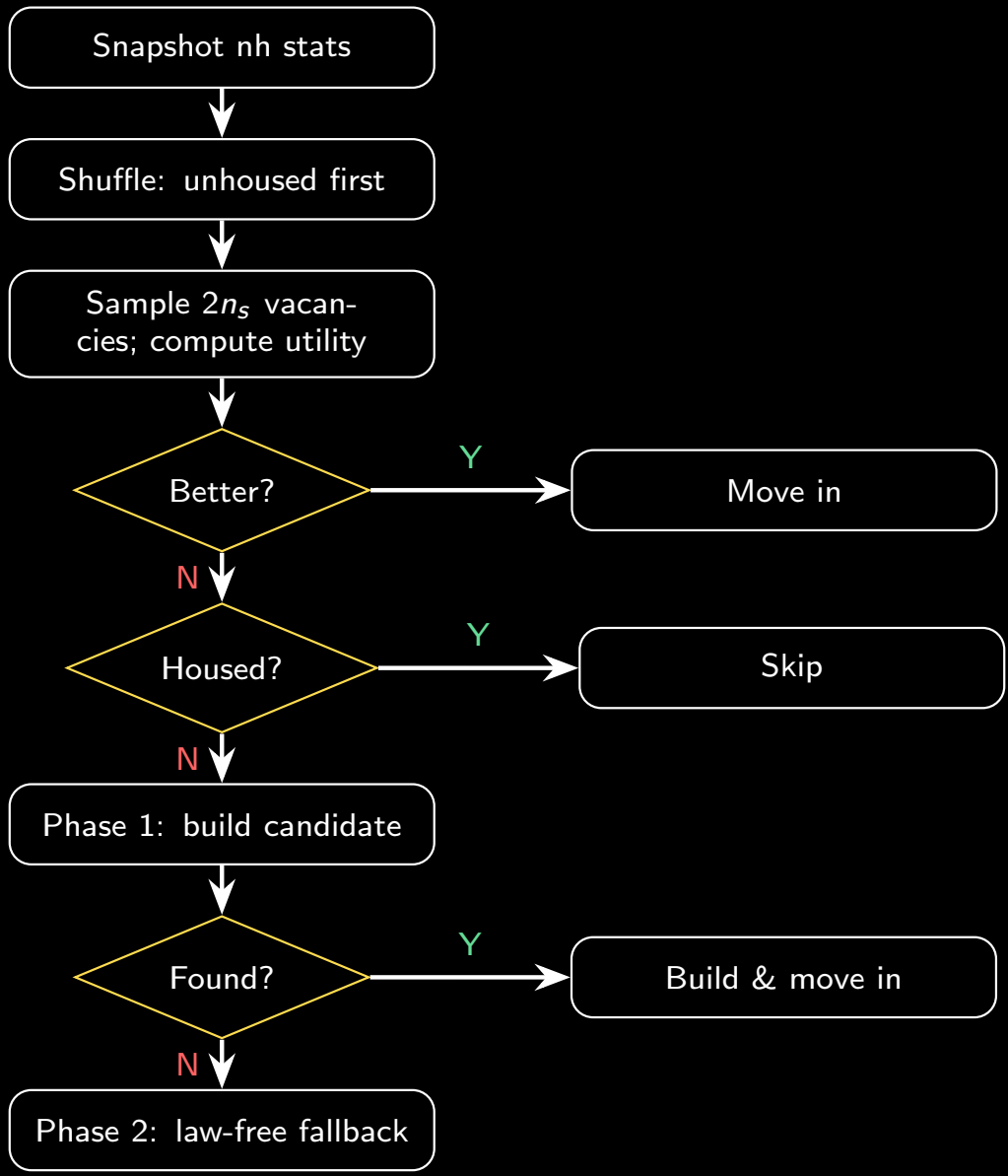
Five marginal utilities $u_k \in [0, 1]$, combined via Frank copula:

$$C_\theta(u_1, \ldots, u_5) = -\frac{1}{\theta} \ln\left(1 + \frac{\prod_k (e^{-\theta u_k} - 1)}{(e^{-\theta} - 1)^4}\right)$$



$\theta > 0$: positive dependence — all dimensions must be satisfied jointly.

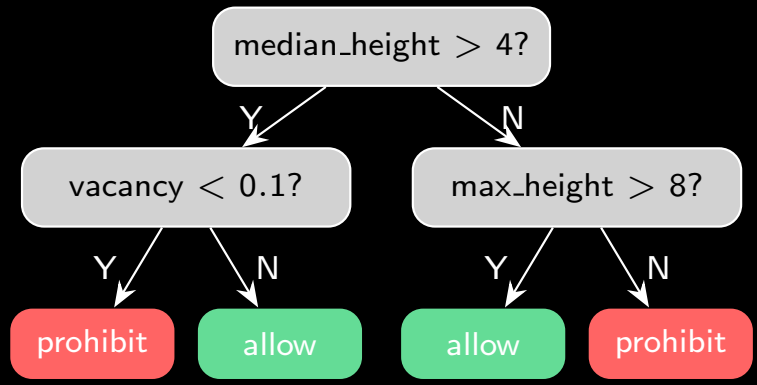
Simulation Step



The NIMBY Mechanism

Every T steps:

1. Propose a random depth-2 decision tree over neighborhood observables
2. Deep-copy city: with law vs without law
3. Run H steps of new inflow on each branch (same RNG seed)
4. Incumbent residents compare utility across branches
5. Majority vote: if $> 50\%$ prefer the law branch $\Rightarrow$ law adopted



Laws are **conditional**: the same law may allow building in one period, block it in another.

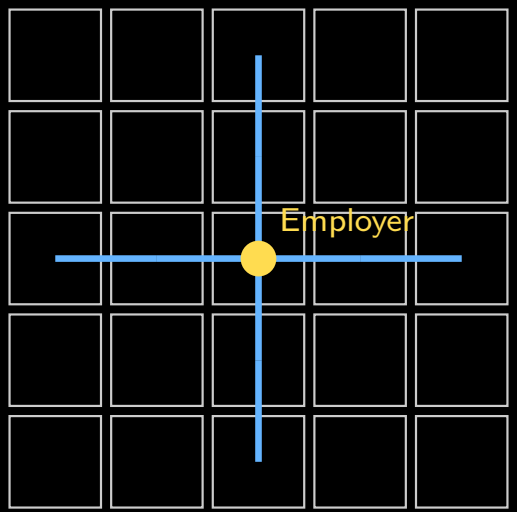
Transport Network & Employer

Roads (BFS hop cache):

- Neighborhoods connected by a road network
- Effective distance $= \min(\text{Euclidean},\ d_{\text{subway}})$
- Subway = walk to hub + hops $\times$ road unit + walk from hub
- Greedy road planner adds highest-contrast edges

Employer (branch simulation):

- Single employer tracks all worker IDs
- Every T_e steps: sample k candidate buildings
- Deep-copy and run H_e steps on each candidate
- Relocate to minimise mean effective commute



What Emerges

1. **Sorting** — agents self-segregate by density preference
2. **Height cascades** — successful tall buildings attract similar agents, law proposals struggle
3. **Law persistence** — once a restrictive law passes, neighborhood stays low-density
4. **Employer drift** — job center migrates toward high-density worker clusters
5. **Unhoused fraction** — regulated cities produce persistent homelessness even with empty land

Building with Generative AI

≈1,500 lines of Julia + Python + HTML across 10 files

Real-time WebSocket visualization, branch-simulated voting, BFS road cache

Built over ∼9 sessions with Claude as collaborator

How AI Was Used

Architecture

- Struct design (City, Agent, Law, Employer)
- WebSocket protocol between Julia $\leftrightarrow$ Blender $\leftrightarrow$ browser
- Frank copula parameterisation

Implementation

- Boilerplate (BFS, JSON serialisation)
- Debugging async race conditions
- Blender mesh geometry API

What AI does not replace

- Model design choices (what *should* agents optimize?)
- Identifying economically meaningful bugs
- Interpreting simulation output
- Deciding which parameters to sweep

A Concrete Example: The Employer Bug

1. Employer initialized at building ID 1 — grid corner $(0, 0)$
2. All 5,000 agents assigned job there
3. Agents clustered at corner; single tower $>$ 200 floors
4. AI spotted the bug when shown the output: "employer should start at the city centroid"
5. Fix: compute nearest building to $\left(\frac{n_x \cdot k}{2}, \frac{n_y \cdot k}{2}\right)$

The bug was *economically obvious* in the output but invisible in the code. AI found it by reasoning about *intent*, not just syntax.

What Worked and What Required Care

Worked well

- Iterative refinement: "make roads thin and black"
- Explaining intent, not code
- AI as a persistent memory of the codebase
- Catching off-by-one and async bugs

Required human judgment

- Verifying economic logic (copula signs, utility direction)
- AI over-engineers without constraints
- Hallucinated API calls (Blender Python)
- Deciding what the model is *for*

AI accelerates implementation; it does not replace the researcher.

The Research Travelogue: Kepler

Kepler's Astronomia Nova (1609):

- Introduces elliptical planetary orbits
- *Includes* every false start, dead end, and discarded hypothesis
- Reads as a journey, not a textbook

Most books and papers present a single clean,
pedagogical order — the travelogue is hidden.

The travelogue can be valuable
but is difficult to read.

The Research Travelogue: `AIguide.md`

The `AIguide.md` as modern travelogue:

- Records each session with Claude — design intent, not just code
- Includes pivots, bugs, and discarded approaches
- The model as built *cannot* be recovered from the code alone

Just as *Astronomia Nova* shows how Kepler *arrived* at ellipses, the AI session log shows how the model arrived at its current design.

AI-assisted research may routinely produce travelogue artifacts — a new kind of research record worth preserving.

Research Questions for the Sweep

1. Does a longer voting horizon (H) produce more or less restrictive cities?
2. What is the unhoused fraction as a function of law frequency (T)?
3. How does city density ($n_{\sqrt{\ }}$) shape law adoption and housing outcomes?
4. Does a road network change zoning dynamics?

Parameter Sweep Design

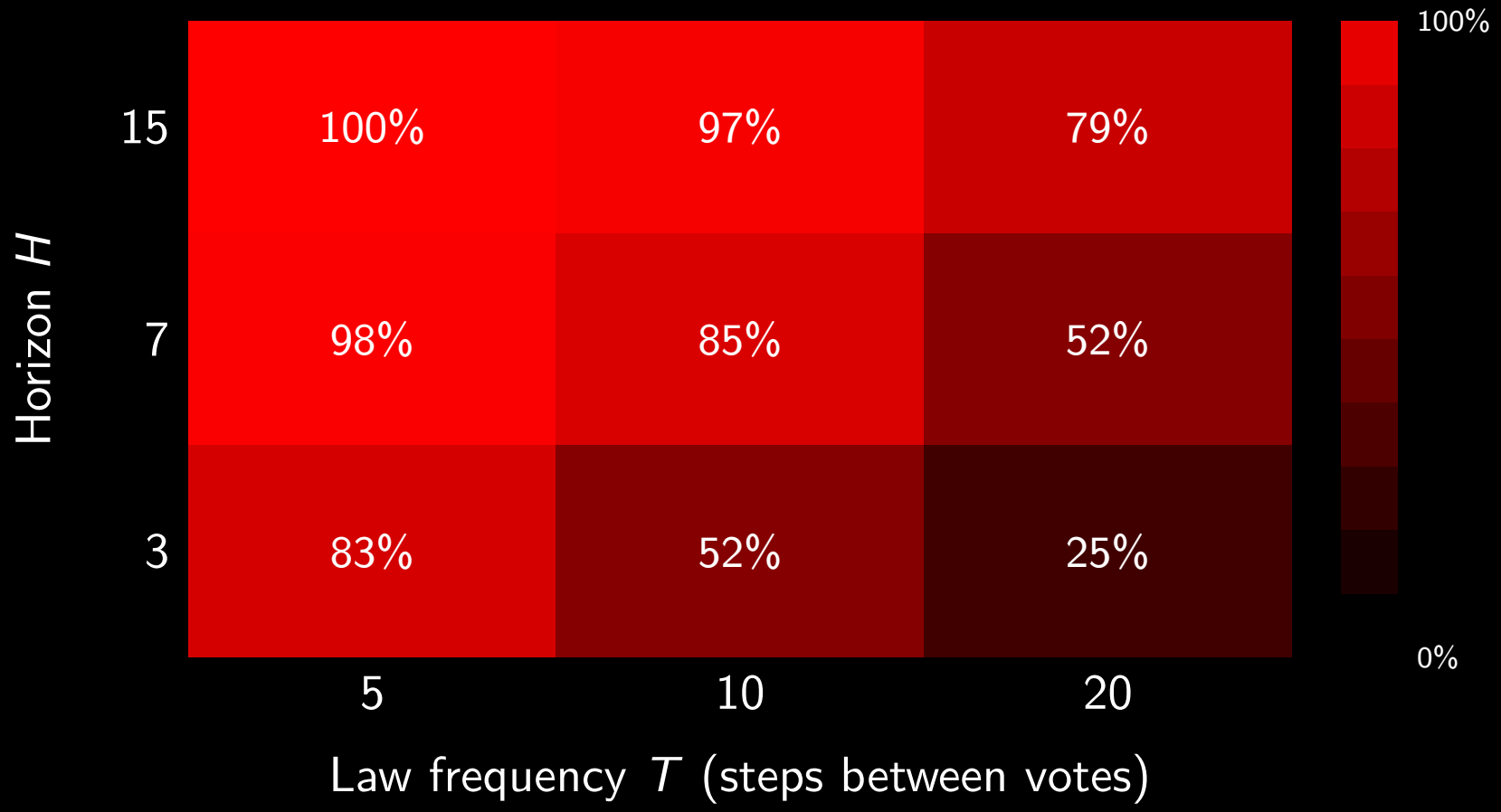
Parameter	Meaning	Values
`enable_roads`	transport network on/off	false, true
T	law eval every N steps	5, 10, 20
H	law eval horizon (look-ahead)	3, 7, 15
$n_{\sqrt{\ }}$	city side (neighborhoods)	6, 8, 10
seed	replication	42, 137, 999

$2 \times 3 \times 3 \times 3 \times 3 = \mathbf{162}$ configurations, $\times$ 100 steps each.

Population **fixed**: 500 initial + 50 inflow/step across all city sizes $\Rightarrow$ density varies with $n_{\sqrt{\ }}$.

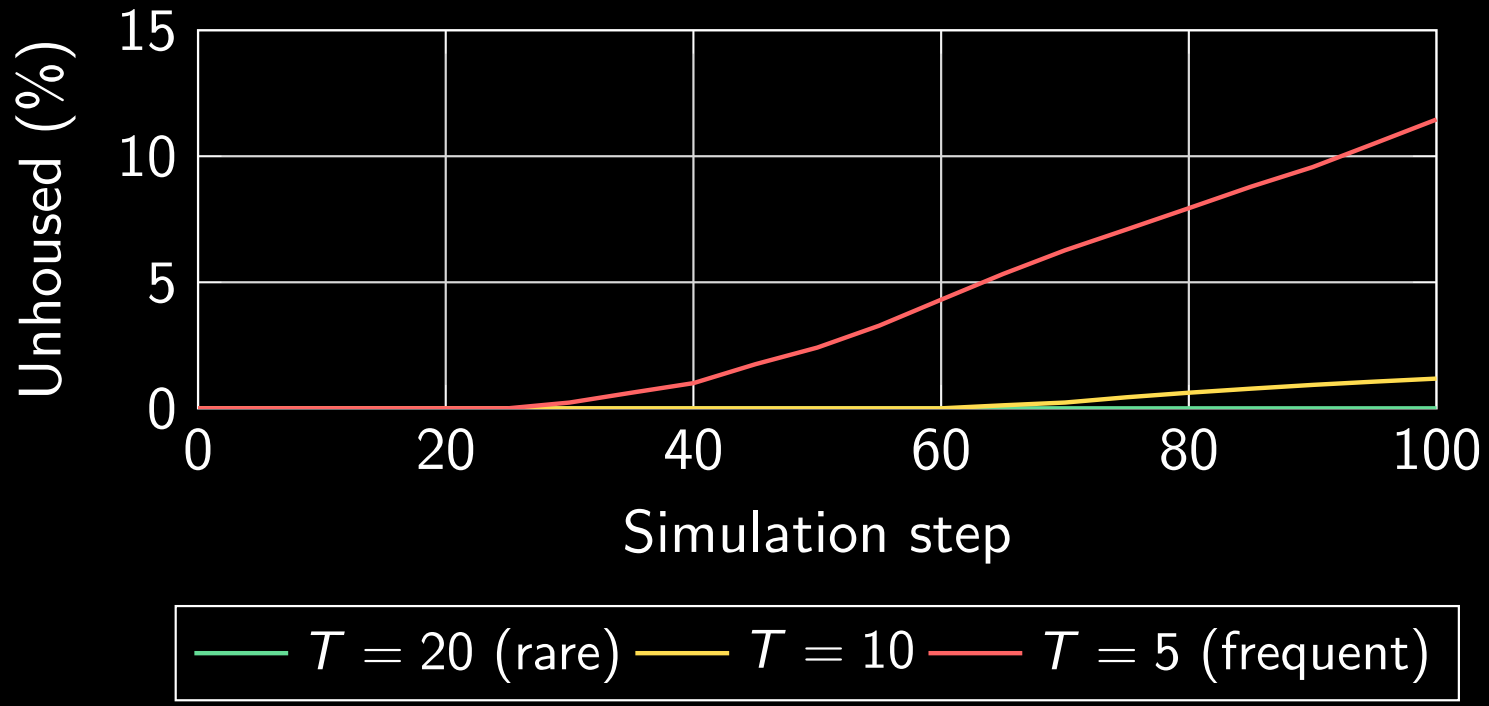
Outcome variables: % neighborhoods with active law, mean building height, unhoused fraction.

Law Adoption: % Neighborhoods with Active Law (Step 100)



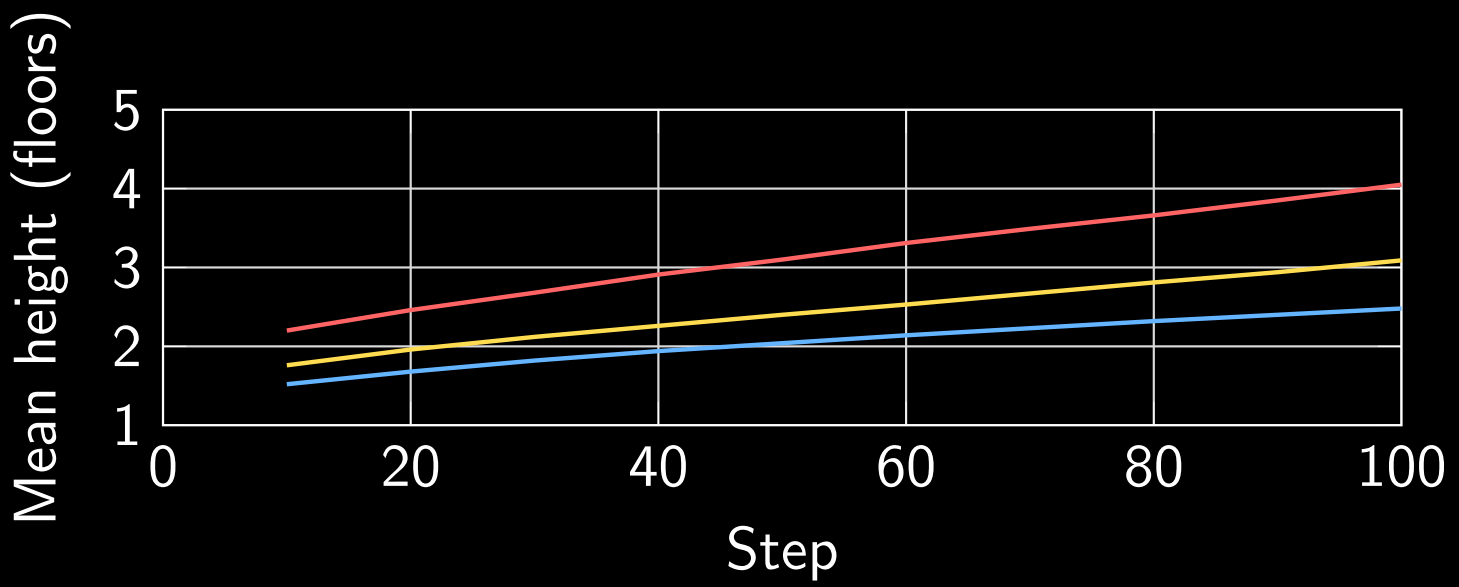
Longer horizon H and more frequent votes (smaller T) $\Rightarrow$ more restrictive cities. Averaged over city size and seed ($n = 18$ per cell).

Unhoused Fraction vs. Law Frequency



Averaged over $H \in \{3, 7, 15\}$, $n_{\surd} \in \{6, 8, 10\}$, seeds, roads ($n = 27$/curve). $T = 20$: zero unhoused throughout. $T = 5$: 11.5% unhoused by step 100.

City Size Effect: Density Amplifies Law Formation



Population fixed (500 init + 50/step) — density varies with n.

City	% laws @ step 100	Unhoused %	Mean height
$n = 6$ (crowded)	83.0%	10.2%	4.05
$n = 8$	76.2%	2.0%	3.09
$n = 10$ (sparse)	64.4%	0.4%	2.48

Connection to the Real Market

1. **NIMBY** mechanisms protect incumbent interests at the neighborhood scale:
   incumbents vote to restrict new supply, preserving neighborhood character and property values
2. This produces supply restriction $\Rightarrow$ elevated prices + reduced volume
3. ABM captures the feedback: laws slow supply $\Rightarrow$ prices rise $\Rightarrow$ incumbents have more to protect $\Rightarrow$ laws tighten
4. Fairfax County is a natural calibration target: documented high regulatory burden, post-2021 price stickiness

Zoning Laws Are Already Trees

An experiment: fed the full Fairfax County zoning ordinance to ChatGPT and asked:

"To what extent can these rules be represented as decision trees?"

- **Most can** — permit thresholds, setbacks, height limits, and use-by-right tables all branch on observable lot and neighborhood attributes
- **Main exception:** special-purpose and planned-development districts, which require case-by-case negotiation

Why this matters computationally:

- Zone types map naturally to Julia **structs**
- **Multiple dispatch** lets different law types share a single `can_build_in_neighborhood` interface — no `if/else` towers
- Opens a path to a **synthetic Fairfax County**: parameterise the model directly from the ordinance, match the observed height and vacancy distribution

Future Work

Model extensions

- Heterogeneous employers (job clusters, not a single center)
- Realtor agent coordinating buyer/seller matching
- Demographic aging (cohort-based entry/exit)

Calibration

- Fairfax County parcel data $\Rightarrow$ building height distribution
- HMDA data $\Rightarrow$ agent budget distribution
- Zoning board records $\Rightarrow$ law adoption frequency
- Moment-matching on median price trajectory

Summary

1. **Empirically**: NIMBY zoning explains post-2022 price stickiness in tight markets
2. **The ABM**: heterogeneous agents sort, build, vote on laws, and drive employer location — rich emergent dynamics
3. **AI as collaborator**: accelerates implementation; human judgment still required for model design and interpretation
4. **Parameter sweeps**: frequent votes ($T = 5$) drive 11.5% unhoused; longer horizon H and crowded cities ($n = 6$) push law adoption to $\sim$100%
5. **Next step**: deepen the bridge between model mechanics and real urban policy

Thank You

John S. Schuler

jschule4@gmu.edu

Department of Computational and Data Sciences
George Mason University

Code: `github.com/jsschuler/urban1`

Questions welcome on: ABM design · AI-assisted research · parameter sweep results